DOCUMENTATION TECHNIQUE DU FRONTEND

1. VISION D'ENSEMBLE ET REPERTOIRE SOURCE

Le frontend de DigiMarket est conçu comme une interface de consommation pour l'API REST Flask, dédiée à la vente de composants PC. L'objectif architectural est de dissocier la logique de présentation de la persistance des données, tout en utilisant Flask comme moteur de rendu serveur pour optimiser le SEO et le temps de chargement initial.

Les ressources clés du projet sont structurées comme suit dans le dépôt Git (https://github.com/JeanRosselVallee/BlentPythonProject.git) :

- Dossier des templates : app/templates/
- Contrôleur web (Blueprints) : app/web/routes.py
- Point d'entrée de l'application : run.py

2. STRATEGIE DE TEMPLATING ET HIERARCHIE VISUELLE

Le projet adopte une stratégie d'héritage de templates via layout.html. Ce choix répond au principe **DRY (Don't Repeat Yourself)** : il permet de centraliser la gestion des métadonnées, des feuilles de style et des scripts globaux. Pour l'architecte, cette structure facilite la maintenance évolutive, car toute modification du header ou du footer est immédiatement répercutée sur l'ensemble des vues.

Structure de base (layout.html) :

```
<!-- Base layout for global assets and SEO -->
<html>
  <body>
    {% block content %}{% endblock %}
    {% block scripts %}{% endblock %}
  </body>
</html>
```

Les composants UI critiques identifiés pour assurer l'interactivité sont les suivants :

Élément UI / Fonction	Rôle Technique
#cart-total	Identifiant DOM dédié à la mise à jour dynamique du montant total via JavaScript.
showCheckout()	Fonction orchestrant la transition vers le tunnel d'achat et la validation de commande.

3. FLUX DE DONNEES : DE FLASK A JINJA2

Actuellement, le contrôleur app/web/routes.py fait office de "coquille" de rendu : il appelle render_template() sans passer d'arguments, car l'injection dynamique (ex: products) est pour l'instant commentée dans le code source. La structure cible prévoit d'injecter les modèles SQLAlchemy directement dans le contexte Jinja2 pour un rendu serveur performant.

Modèle d'itération cible dans catalog.html :

```
<!-- Scalable product listing using Jinja2 loop -->
{% for product in products %}
<div class="product-card">
    <h3>{{ product.nom }}</h3>
    <p>{{ product.prix }} €</p>
    <button onclick="addToCart({{ product.id }})">Add to Cart</button>
</div>
{% endfor %}
```

4. LOGIQUE METIER ET GESTION DES ÉTATS COMPLEXES

GESTION DU PANIER (JAVASCRIPT)

Le panier repose sur une gestion d'état côté client. Pour éviter la perte de données lors du rafraîchissement des pages, les états (ID produits, quantités) doivent être persistés dans le localStorage. Le frontend interagit avec le DOM pour refléter les quantités et recalculer le montant total affiché dans #cart-total sans rechargement de page.

AUTHENTIFICATION ET REDIRECTION

Lors d'un appel réussi à /api/auth/login, le serveur retourne un jeton JWT sous la clé token. Ce jeton est stocké localement pour authentifier les requêtes futures vers les routes protégées (comme /api/commandes). La redirection est ensuite opérée vers /catalog ou /orders selon le succès de l'opération.

Logique d'affichage basée sur le rôle :

```
<!-- Role-based UI logic for admin privileges -->
{% if role == 'admin' %}
   <a href="/admin">Admin Panel</a>
{% else %}
   <a href="/orders">My Orders</a>
{% endif %}
```

5. INTEGRATION DES SERVICES ET ENDPOINTS API

Le frontend consomme les services exposés par les Blueprints backend. Notez que l'accès aux commandes nécessite l'inclusion du JWT dans les headers HTTP (via le décorateur @requires_authorization côté serveur).

Page Frontend	Endpoint API Consommé	Action Technique
catalog.html	GET /api/produits	Récupération de la liste publique des composants.
orders.html	GET /api/commandes	Affichage de l'historique (authentification requise).
Achat final	POST /api/commandes	Création de commande avec validation de stock.

6. GUIDE DE MAINTENANCE ET SCALABILITE

Pour garantir la pérennité du système frontend, les recommandations suivantes doivent être appliquées :

- **Isolation des scripts :** Toute logique JavaScript spécifique à une vue doit impérativement être encapsulée dans le bloc {% block scripts %} pour éviter de polluer l'espace global et garantir un chargement post-DOM.
- **Identifiants Uniques :** Le respect strict des IDs (#cart-total, etc.) est crucial pour la stabilité des sélecteurs JavaScript.

- **Validation des Types de Données :** Jinja2 échouera silencieusement ou provoquera des erreurs de rendu si une variable attendue est None. Il est impératif de valider ou d'initialiser les variables (ex: passer une liste vide [] au lieu de None) dans app/web/routes.py avant l'appel à render_template.